

Introdução à Programação na Web / Programação na Internet

Inverno de 2025/2026

* Teste Global de Época Normal (14 de Janeiro de 2026) - Duração 1h30

NÚMERO _____ NOME _____

GRUPO 1 [4 valores]

Nas seguintes questões, selecione a única opção verdadeira, escrevendo-a na folha de teste. Uma resposta correta conta a totalidade da cotação da pergunta; uma resposta incorreta desconta 25% da cotação da pergunta ao total do grupo; uma questão não respondida conta 0 valores.

1. Ao analisar o URI `http://localhost:8080/products?limit=3`, qual das seguintes afirmações é verdadeira?
 - A. A query string neste URL é `limit=3`
 - B. O caminho neste URL é `http://localhost:8080`
 - C. A query string neste URL é `/products?limit=3`
 - D. O caminho neste URL é `limit=3`
2. Qual das opções descreve corretamente o header HTTP Content-Type?
 - A. Indica o tipo de autenticação usado apenas em respostas HTTP.
 - B. Define o formato dos dados enviados apenas em pedidos HTTP.
 - C. Especifica o tipo de conteúdo do corpo da mensagem e pode ser usado tanto em pedidos como em respostas HTTP.
 - D. Serve para indicar o idioma preferido do cliente no pedido HTTP.
3. O que significa o status code HTTP 400 (Bad Request)?
 - A. O servidor encontrou um erro interno inesperado.
 - B. O pedido foi bem-sucedido, mas não há conteúdo para retornar.
 - C. O recurso solicitado foi movido permanentemente.
 - D. O servidor não conseguiu processar o pedido porque este contém dados inválidos.
4. Numa aplicação Express, `app.use(express.json())` serve para:
 - A. Servir ficheiros estáticos
 - B. Fazer o parse do body dos pedidos recebidos cujo o body está no formato JSON
 - C. Converter respostas para JSON
 - D. Fazer o parse do body das respostas cujo o body está no formato JSON

5. Considere o código Javascript e indique a opção verdadeira:

```
console.log("Início");  
let flag = true  
setTimeout(() => { flag=false}, 1000);  
while (flag);  
console.log("Fim");
```

- A. O `setTimeout` consegue alterar a variável `flag` e o loop termina após de 1 segundo.
 - B. O `while(flag);` bloqueia o event loop indefinidamente
 - C. O `console.log("Fim")` é exibido imediatamente após `console.log("Início")`
 - D. O Node.js interrompe automaticamente o loop após 1 segundo
6. Considere o seguinte trecho de um ficheiro YAML/JSON representando uma especificação OpenAPI. Qual das seguintes alternativas descreve corretamente um exemplo de pedido HTTP para o endpoint apresentado?
 - A. GET `/users/?xyz=1`
 - B. GET `/users?xyz=abc`
 - C. GET `/users/xyz`
 - D. GET `/users/xyz/1`

```
paths:  
  /users:  
    get:  
      parameters:  
        - name: xyz  
          in: query  
          required: true  
          schema:  
            type: integer
```

```
"paths": {  
  "/users": {  
    "get": {  
      "parameters": [  
        {  
          "name": "xyz",  
          "in": "query",  
          "required": true,  
          "schema": {  
            "type": "integer"  
          }  
        }  
      ]  
    }  
  }  
}
```

Introdução à Programação na Web / Programação na Internet

Inverno de 2025/2026

* Teste Global de Época Normal (14 de Janeiro de 2026) - Duração 1h30

NÚMERO _____ NOME _____

GRUPO 2 [4 valores]

Faça a correspondência entre o que está no lado esquerdo da tabela (identificado com números) e o que lhe está relacionado no lado direito da tabela (identificado por letras). Se por exemplo considerar que a opção 1 do lado esquerdo está relacionada com a opção c do lado direito, responda 1-c. Na folha de teste.

1. Na linguagem HTML:

1	O elemento DIV	a	contém todo o conteúdo visível de um documento HTML
2	O atributo href de um elemento <a>	b	é um contentor de elementos HTML
3	O elemento BODY	c	especifica o URL de um hyperlink
4	O atributo action do elemento <form>	d	especifica o URL para onde os dados do formulário são enviados

2. No contexto de Cascading Style Sheets (CSS) associe:

1	Seletor de class	a	#redInput
2	Seletor de element	b	boldText
3	Seletor de id	c	input
4	Não é um seletor CSS	d	.redInput

3. A resposta HTTP para o URL apresentado no seguinte código Javascript tem no corpo uma mensagem JSON que representa um objeto com as propriedades *question* e *answer*, ambas strings.

```
fetch("http://www.jokes.com/random_joke")  
  .then(r => r.json()) // 1  
  .then(joke => {console.log(joke.question); return joke.answer}) // 2  
  .then(a => setTimeout(() => {console.log(a)}, 3000)) // 3  
  .then(_ => console.log("Waiting...")) // 4  
  .catch(err => {console.error(`Error: ${err}`)});
```

1	O fetch retorna	a	uma promise de um Response
2	r.json() retorna	b	uma promise de um objeto
3	O último then retorna	c	uma promise de undefined
4	O segundo then retorna	d	uma promise de uma string

4. Numa aplicação Express em Node.js, o lado esquerdo corresponde aos pedidos HTTP e o lado direito corresponde às rotas associadas aos pedidos.

1	GET /users/42/profile	a	app.get("/", funcA)
2	GET /administrators	b	app.get("/users/:status", funcB)
3	GET /users/active?admin=true	c	app.get("/:group", funcC)
4	GET /?name=filipe	d	app.get("/users/:id/:filter", funcD)

5. No objeto Response do Express:

1	A propriedade status	a	define um header HTTP na resposta enviada ao cliente
2	O método set	b	envia o body da resposta no formato JSON
3	O método json	c	especifica o status code HTTP da resposta
4	O método redirect	d	envia uma resposta HTTP com status code 3xx e o header Location com o URL passado por parâmetro

6. Nos módulos ECMAScript:

1	export default function getName() {}	a	Exporta valores nomeados de um módulo
2	export { name, age }	b	Importa exportações específicas de um módulo
3	import * as utils from './utils.js'	c	Importa os símbolos exportados pelo módulo num objeto
4	import { name } from './user.js'	d	Exporta uma função como exportação padrão

NÚMERO _____ NOME _____

GRUPO 3 [3.5 valores]

[3.5] Considere o seguinte código em JavaScript.

<pre>1 const start = Date.now(); 2 function wait(ms) { 3 return new Promise(resolve => 4 setTimeout(() => resolve(), ms)); 5 } 6 function getTimestamp() { 7 return ((Date.now()-start)/1000).toFixed(0) +"s"; 8 } 9 function log(message) { 10 console.log(`[\${getTimestamp()}] \${message}`); 11 }</pre>	<pre>12 async function processOrder() { 13 log('recebido'); 14 await wait(1000); 15 log('Pedido validado'); 16 await wait(2000); 17 log('Pedido processado'); 18 await wait(1000); 19 log('Pedido concluído'); 20 } 21 processOrder(); 22 log('Sistema disponível');</pre>
--	---

- a. [1] Qual o output do programa?
- b. [2.5] Crie uma versão alternativa utilizando apenas .then() em vez de async/await, que apresente o mesmo resultado e nos mesmos tempos que a implementação anterior.

GRUPO 4 [3.5 valores]

[3.5] Considere o módulo functionRegistry, que permite registar e executar funções assíncronas identificadas por uma chave. Escreva o código que falta na implementação, indicando o número correspondente ao comentário existente no final de cada linha incompleta.

A função register(key, fn) regista uma função assíncrona fn associada à chave key, tornando-a disponível para execuções posteriores, através dessa chave.

A função execute(key) executa a função assíncrona associada à chave key. Se a função já estiver em execução (estado *running*), retorna a Promise em curso. Caso contrário, inicia a execução e coloca-a no estado *running*.

<pre>1 import functionRegistry from "./function-registry.mjs"; 2 3 function asyncJob() { 4 return new Promise((resolve) => { 5 setTimeout(() => { 6 resolve("Job done"); 7 }, 1000); 8 }); 9 } 10 const registry = functionRegistry(); 11 12 registry.register("jobA", () => asyncJob()); 13 14 const p1 = registry.execute("jobA"); 15 const p2 = registry.execute("jobA"); 16 17 console.log(p1 === p2); // true 18 19 console.log(await p1); // Job Done 20 21 const p3 = registry.execute("jobA"); 22 console.log(p1 === p3); // false</pre>

Introdução à Programação na Web / Programação na Internet

Inverno de 2025/2026

Teste Global de Época Normal (14 de Janeiro de 2026) - Duração 1h30

NÚMERO _____ NOME _____

function-registry.mjs	
1	export default function functionRegistry() {
2	
3	const registry = _____; //1
4	
5	function register(key, fn) {
6	if(registry[key])
7	throw new Error(`Function with key "\${key}" is already registered`);
8	_____ = { fn, running: false, promise: null }; // 2
9	}
10	
11	function execute(key) {
12	const entry = registry[key];
13	
14	if (_____) throw new Error(`Function with key "\${key}" is not registered`); // 3
15	if (entry.running) return _____; // 4
16	
17	entry.running = true;
18	const promise = entry.fn()
19	.then((result) => {
20	entry.running = false;
21	return _____; // 5
22	})
23	
24	entry.promise = promise;
25	return _____; // 6
26	}
27	
28	return { register, execute };
29	}

GRUPO 5 [5 valores]

[5] Desenvolva uma aplicação em Node.js com Express, que armazena em memória o conteúdo do corpo (req.body) dos últimos n pedidos HTTP recebidos, sendo n configurável. Quando o limite n é ultrapassado, o body mais antigo é removido. A aplicação mantém o array storedRequests e implementa um middleware global que, para cada pedido com body presente, adiciona-o ao array.

A aplicação deve expor duas rotas:

1. **/stored**
 - Retorna todos os conteúdos dos pedidos armazenados em formato JSON.
 2. **/stored/:idx**
 - Recebe na *path* o parâmetro *idx* que indica o índice do pedido guardado a retornar. Assim, quando o *idx* é 0 retorna o pedido mais recente/último guardado, quando for 1, o penúltimo, e assim sucessivamente.
- a) [1] Complete a implementação do módulo service.mjs.
b) [2] Complete a implementação do módulo middleware.mjs.
c) [2] Complete a implementação do módulo server.mjs.

NÚMERO _____ NOME _____

service.mjs		middleware.mjs	
1	const storedRequests = [];	1	import { addRequest } from "../services.mjs";
2	export function addRequest(body, n) {	2	
3	_____ // 1	3	function createFilterMiddleware(n) {
4	if (_____) { // 2	4	return (req, res, next) => {
5	// Removes the first element of the array	5	const body = _____ // 1
6	storedRequests.shift();	6	if (body && typeof body === "object") {
7	}	7	_____ // 2
8	}	8	} _____ // 3
9	export function getAllRequests()	9	};
10	{_____} // 3	10	}
11	export function getRequest(idx) {	11	export default createFilterMiddleware;
12	return storedRequests[idx];		
	}		

server.mjs	
1	import express from "express";
2	import createFilterMiddleware from "../middleware.mjs";
3	import { getAllRequests, getRequest } from "../services.mjs";
4	
5	const app = express();
6	const port = 3000;
7	
8	app.use(express.json());
9	
10	// In this example the middleware stores the last 2 bodies
11	app.use(_____); // 1
12	
13	_____("/stored", (req, res) => { // 2
14	res.json(getAllRequests());
15	});
16	
17	app.get("/stored/:idx", (req, res) => {
18	const idx = req.params.idx;
19	res.json(_____) // 3
20	});
21	
22	app.post("/store", (req, res) => {
23	return res.json({message: "Stored"});
24	});
25	
26	_____(port, () => { // 4
27	console.log(`Server running http://localhost:\${port}`);
	});

Os docentes,
 Fernanda Passos, Filipe Freitas, José Faustino e Luís Falcão,